

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и Компьютерной техники

Отчёт по лабораторной работе №2
Численное решение нелинейных уравнений и систем
Вариант 1
Вычислительная математика

Выполнил:

Бондаренко Андрей Владимирович

Группа: P3215

Преподаватель:

Наумова Надежда Александровна

Санкт-Петербург 2026 г.

Цель работы:

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов.

Задание:

1 Вычислительная реализация задачи:

Состоит из двух частей и отражается ТОЛЬКО в отчете.

1 часть. Решение нелинейного уравнения

Задание:

1. Отделить корни заданного нелинейного уравнения графически (вид уравнения представлен в табл. 6)
2. Определить интервалы изоляции корней.
3. Уточнить корни нелинейного уравнения (см. табл. 6) с точностью $\epsilon=10^{-2}$.
4. Используемые методы для уточнения каждого из 3-х корней многочлена представлены в таблице 7.
5. Вычисления оформить в виде таблиц (1-5), в зависимости от заданного метода. Для всех значений в таблице удерживать **3 знака** после запятой.
 - 5.1 Для метода половинного деления заполнить таблицу 1.
 - 5.2 Для метода хорд заполнить таблицу 2.
 - 5.3 Для метода Ньютона заполнить таблицу 3.
 - 5.4 Для метода секущих заполнить таблицу 4.
 - 5.5 Для метода простой итерации заполнить таблицу 5. Проверить условие сходимости метода на выбранном интервале.**
6. Заполненные таблицы отобразить в отчете.

2 часть. Решение системы нелинейных уравнений

Задание:

1. Отделить корни заданной системы нелинейных уравнений графически (вид системы представлен в табл. 8).
2. Используя указанный метод, решить систему нелинейных уравнений с точностью до 0,01.
3. Для метода простой итерации проверить условие сходимости метода.
4. Подробные вычисления привести в отчете.

2 Программная реализация задачи:

Для нелинейных уравнений:

1. Все численные методы (см. табл. 9) должны быть реализованы в виде отдельных подпрограмм/методов/классов.
2. Пользователь выбирает уравнение, корень/корни которого требуется вычислить (3-5 функций, в том числе и трансцендентные), из тех, которые предлагает программа.
3. Предусмотреть ввод исходных данных (границы интервала/начальное приближение к корню и погрешность вычисления) из файла или с клавиатуры по выбору конечного пользователя.
4. Выполнить верификацию исходных данных. Необходимо анализировать наличие корня на введенном интервале. Если на интервале несколько корней или они отсутствуют – выдавать соответствующее сообщение. Программа должна реагировать на некорректные введенные данные.
5. Для методов, требующих начальное приближение к корню (методы Ньютона, секущих, хорд с фиксированным концом, простой итерации), выбор начального приближения x_0 (а или b) вычислять в программе.
6. Для метода простой итерации проверять достаточное условие сходимости метода на введенном интервале.
7. Предусмотреть вывод результатов (найденный корень уравнения, значение функции в корне, число итераций) в файл или на экран по выбору конечного пользователя.
8. Организовать вывод графика функции, график должен полностью отображать весь исследуемый интервал.

Для систем нелинейных уравнений:

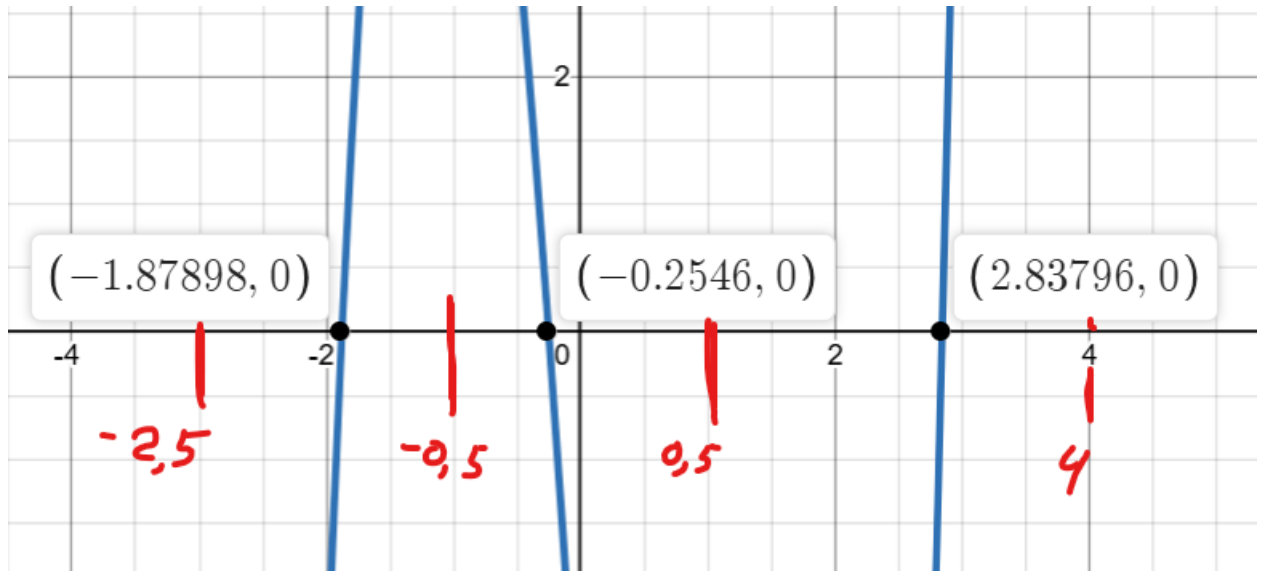
1. Пользователь выбирает предлагаемые программой системы двух нелинейных уравнений (2-3 системы).
2. Организовать вывод графика функций.
3. Начальные приближения ввести с клавиатуры.
4. Для метода простой итерации проверить достаточное условие сходимости.
5. Организовать вывод вектора неизвестных: x_1, x_2 .
6. Организовать вывод количества итераций, за которое было найдено решение.
7. Организовать вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$
8. Проверить правильность решения системы нелинейных уравнений.

1 Вычислительная реализация задачи

1 часть. Решение нелинейного уравнения

Уравнение: $2.74 \cdot x^3 - 1.93 \cdot x^2 - 15.28 \cdot x - 3.72$

Корни: -1.8789, -0.2546, 2.8379



Интервалы изоляции корней

Для левого: (-2.5, -0.5)

Для среднего: (-0.5, 0.5)

Для правого: (0.5, 4)

Рабочие формулы:

Метод половинного деления:

$$c = (a + b) / 2$$

Если $f(a) * f(c) \leq 0$, то $b = c$

Иначе $a = c$

Работаем пока $|b - a| < \varepsilon$

Ответ – a , c или b в зависимости от минимального из $f(a), f(c), f(b)$

Метод половинного деления:

№	a	b	x	f(a)	f(b)	f(x)	a-b
1	2.25	4	3.125	-16.66	79.64	13.301	1.75
2	2.25	3.125	2.688	-16.66	13.301	-5.539	0.875
3	2.688	3.125	2.906	-5.539	13.301	2.83	0.438
4	2.688	2.906	2.797	-5.539	2.83	-1.606	0.219
5	2.797	2.906	2.852	-1.606	2.83	0.547	0.109
6	2.797	2.852	2.824	-1.606	0.547	-0.545	0.055
7	2.824	2.852	2.838	-0.545	0.547	-0.003	0.027
8	2.838	2.852	2.845	-0.003	0.547	0.271	0.014
9	2.838	2.845	2.841	-0.003	0.271	0.134	0.007

Метод простой итерации:

Подберём такую φ чтобы $x = \varphi(x)$

И чтобы выполнялось условие сходимости: $|\varphi'(x)| \leq q < 1$ на интервале изоляции корня

$$\varphi(x) = x + \lambda * f(x)$$

$$\varphi'(x) = 1 + \lambda * f'(x)$$

[a,b]

Пусть $M = \max(|f'(x)|)$ на [a,b], $M \approx 15$

Тогда $\lambda = -1/M \cdot \text{sign}(f'(x))$

Вычисляем: $\lambda \approx 0.066$

Тогда: $\varphi'(x) = 1 - \frac{|f'(x)|}{M} < 1$ т.к. $M \geq |f'(x)|$

Метод простой итерации

№	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	0	-0.246	-0.125	0.246
2	-0.246	-0.254	-0.011	0.008

Метод секущих:

$$\text{Формула: } x_{n+1} = x_n - f(x_n) \cdot \frac{(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

Критерий остановки: $|x_{n+1} - x_n| < \varepsilon$

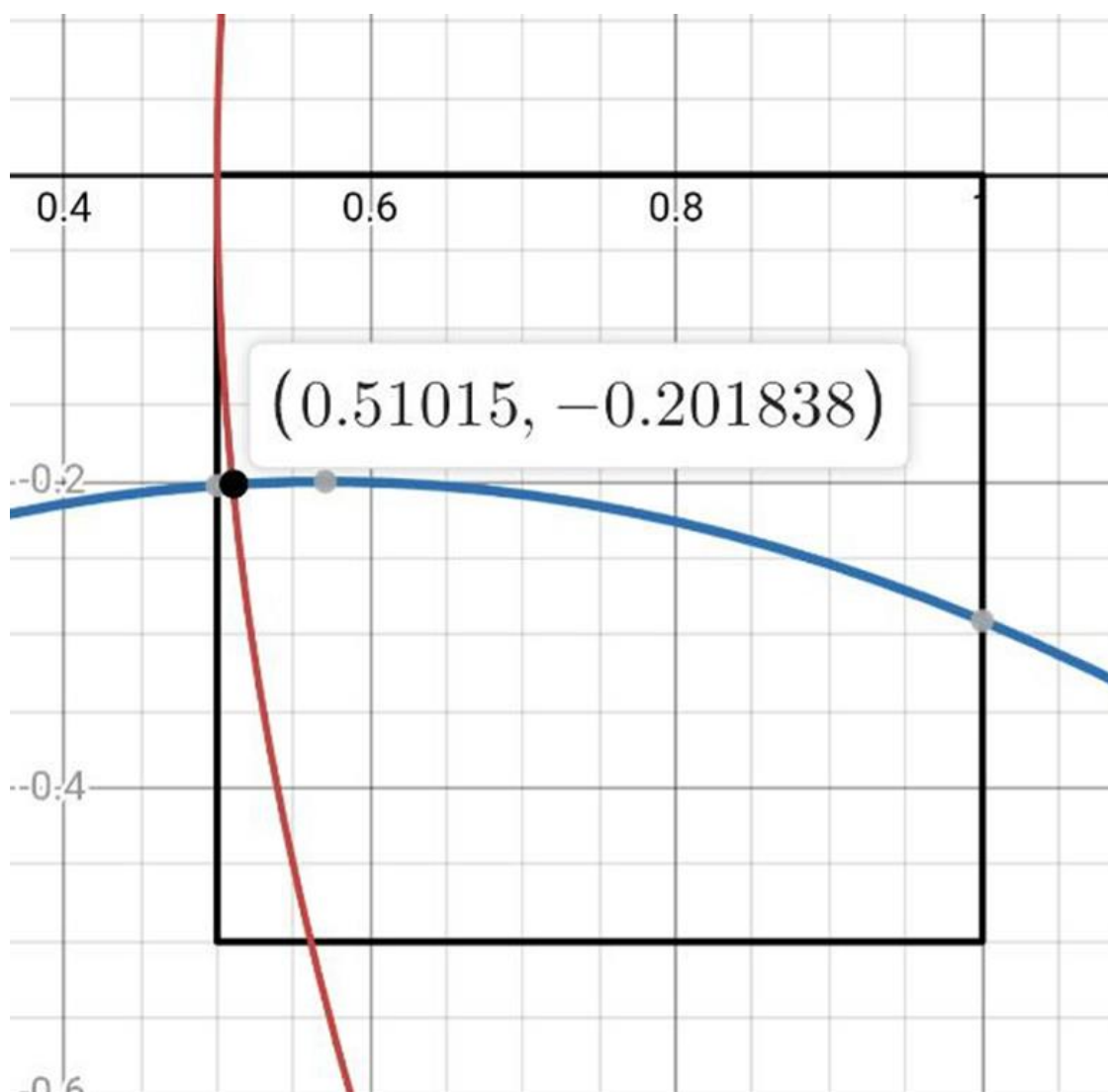
Метод секущих

№	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-2.5	-1.49	-1.711	3.056	0.221
2	-1.49	-1.711	-1.966	-1.951	0.255
3	-1.711	-1.966	-1.866	0.265	0.099
4	-1.966	-1.866	-1.878	0.018	0.012
5	-1.866	-1.878	-1.879	0	0.001

2 часть. Решение системы нелинейных уравнений

Система нелинейных уравнений	Метод
$\begin{cases} \sin(x+1) - y = 1,2 \\ 2x + \cos y = 2 \end{cases}$	Метод простой итерации

Отделим корни графически: $0.5 \leq x \leq 1$; $-0.5 \leq y \leq 0$



Доказательство сходимости

$$\max_{x \in G} \max_i \left| \sum_{j=1}^n \frac{\partial \varphi_i(x)}{\partial x_j} \right| < 1$$

Для $\varphi_1(y) = (2 - \cos y)/2$:

$$\partial \varphi_1 / \partial x = 0$$

$$\partial \varphi_1 / \partial y = (\sin y)/2$$

Для $\varphi_2(x) = \sin(x + 1) - 1.2$:

$$\partial \varphi_2 / \partial x = \cos(x + 1)$$

$$\partial \varphi_2 / \partial y = 0$$

$$\begin{pmatrix} \frac{\partial \varphi_1}{\partial x} & \frac{\partial \varphi_1}{\partial y} \\ \frac{\partial \varphi_2}{\partial x} & \frac{\partial \varphi_2}{\partial y} \end{pmatrix} = \begin{pmatrix} 0 & (\sin y)/2 \\ \cos(x + 1) & 0 \end{pmatrix}$$

$|\sin y| \leq 1$, значит $|\sin y|/2 \leq 0.5 < 1$

$|\cos(x + 1)| \leq 1$ не выполняется в точках $x = \pi k - 1$ но наш интервал не содержит таких точек следовательно достаточное условие выполняется

Метод простой итерации

№	x_k	y_k	$x_{k+1} = \sin(x_k + 1) - 1.2$	$y_{k+1} = (2 - \cos y_k)/2$	$ x_{k+1} - x_k $	$ y_{k+1} - y_k $
1	0.75	-0.25	0.516	-0.216	0.234	0.034
2	0.516	-0.216	0.512	-0.202	0.004	0.014
3	0.512	-0.202	0.51	-0.202	0.002	0

2 Программная реализация задачи:

Для нелинейных уравнений:

Методы в программе: хорд, Ньютона, простой итерации

Код метода хорд:

```
def chord(a, b, eps, f):  
    xp = a  
    k=0  
    while True:  
        k+=1
```

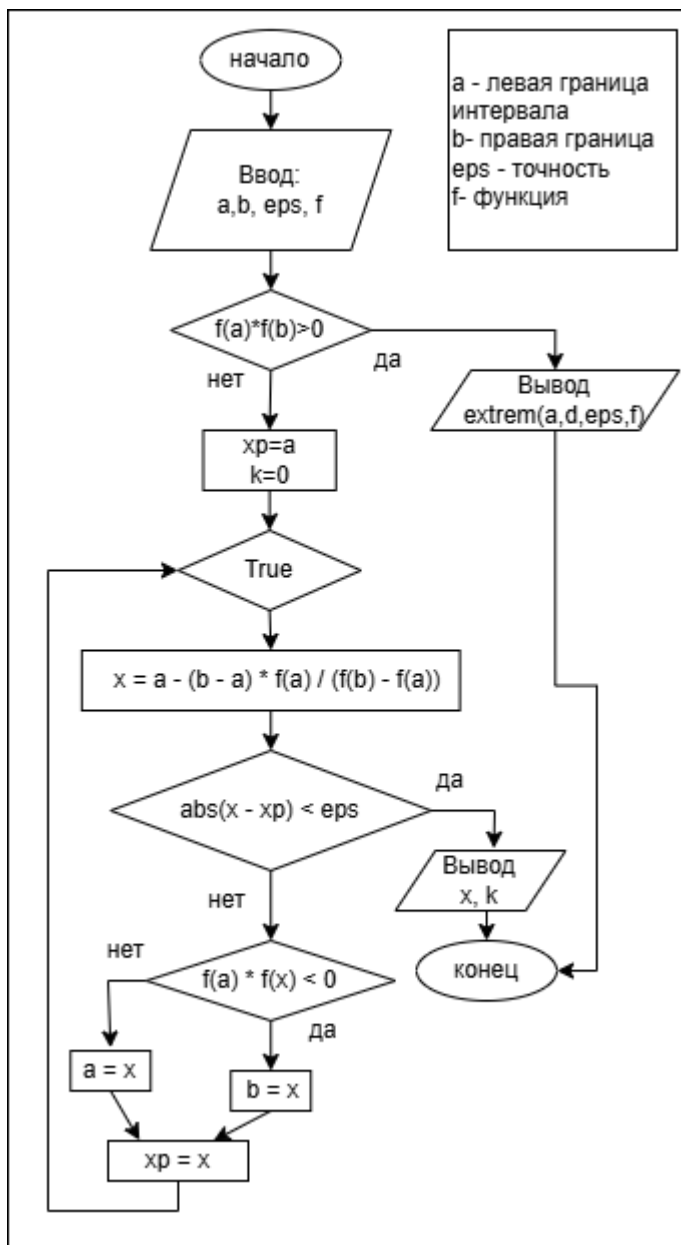
```

x = a - (b - a) * f(a) / (f(b) - f(a))
if abs(x - xp) <= eps:
    return x, k
if f(a) * f(x) < 0:
    b = x
else:
    a = x
xp = x

```

```
def newton(x0, eps, f):
```

Блок-схема:



Код метода Ньютона:

```

def newton(a, b, x0, eps, f):
    x = x0

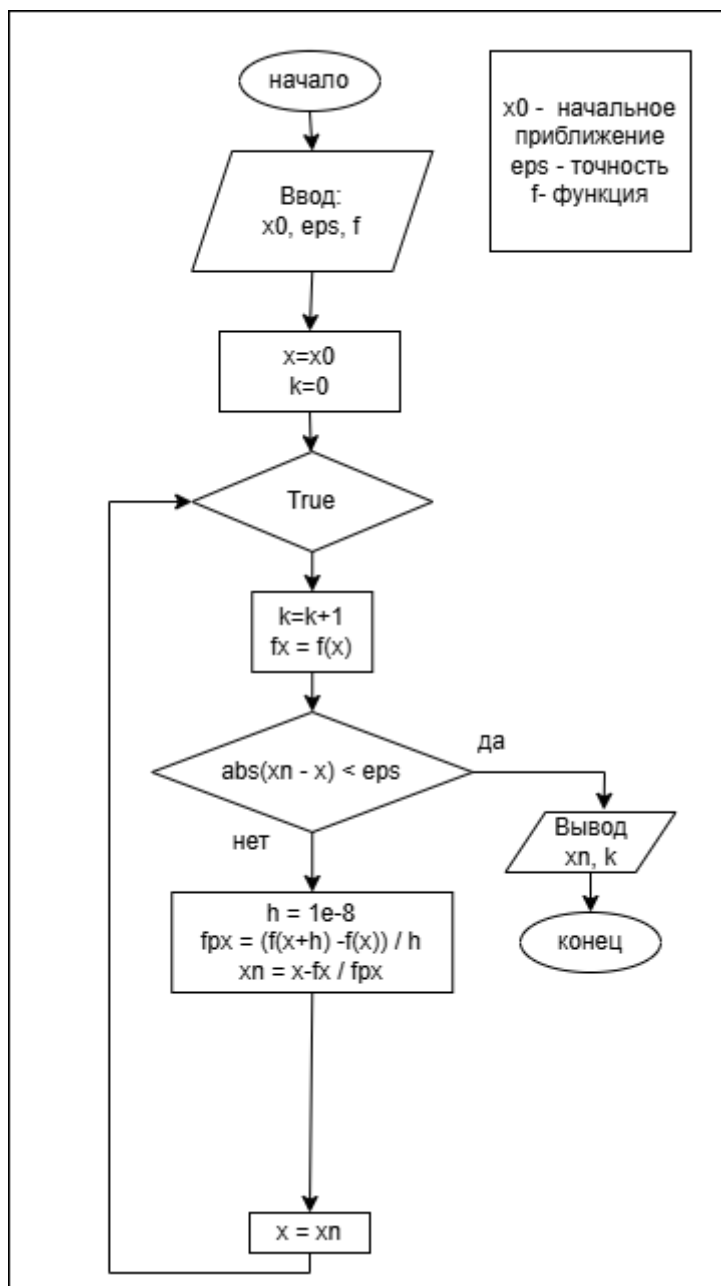
```

```

k=0
while True:
    k += 1
    fx = f(x)
    if (abs(fx) < eps):
        return x, k - 1
    h = max(1e-8, abs(x) * 1e-8)
    fpx = (f(x + h) - f(x - h)) / (h)
    xn = x - fx / fpx
    if xn < a or xn > b:
        xn = a + (b - a) * random.random()
    x = xn

```

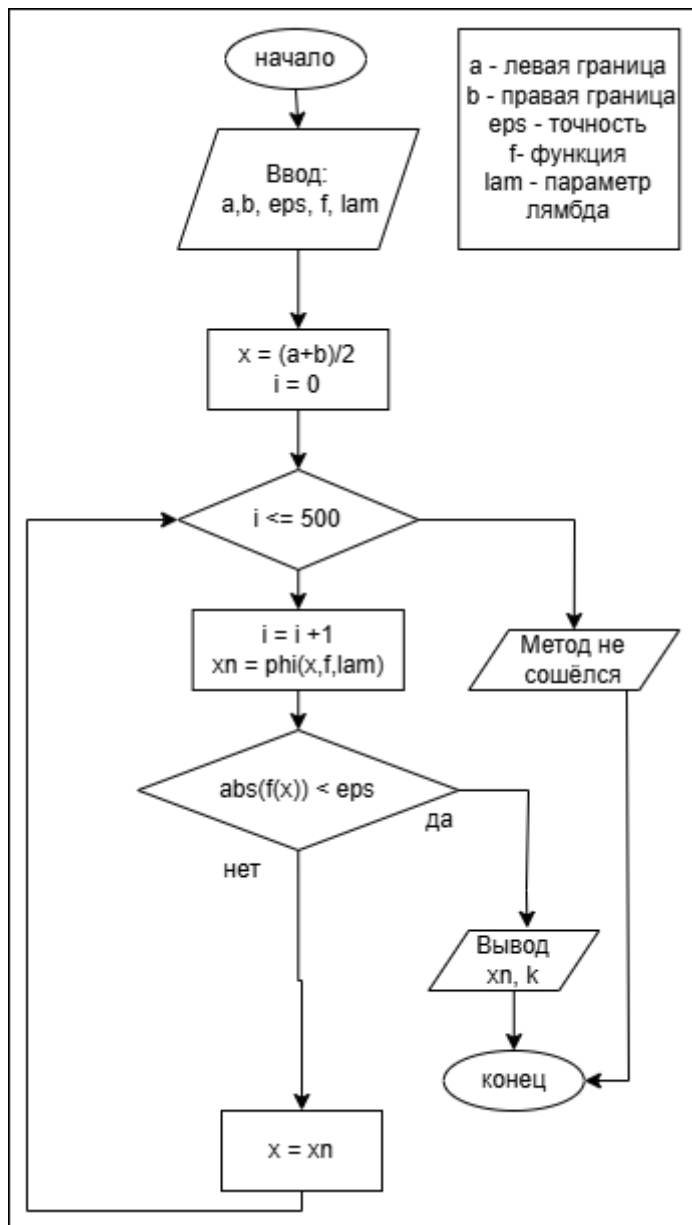
Блок-схема:



Код метода простой итерации:

```
def solve(a,b,eps,f,lam):  
    x = (a + b) / 2  
    i = 0  
    while i <= 500:  
        i += 1  
        xn = phi(x,f,lam)  
        if abs(f(x)) < eps:  
            return xn  
        if xn < a or xn > b:  
            xn = a + (b - a) * random.random()  
        x = xn  
    return "Метод не сошёлся"
```

Блок-схема:



Для систем нелинейных уравнений:

Методы в программе: Метод Ньютона

Код метода Ньютона:

```

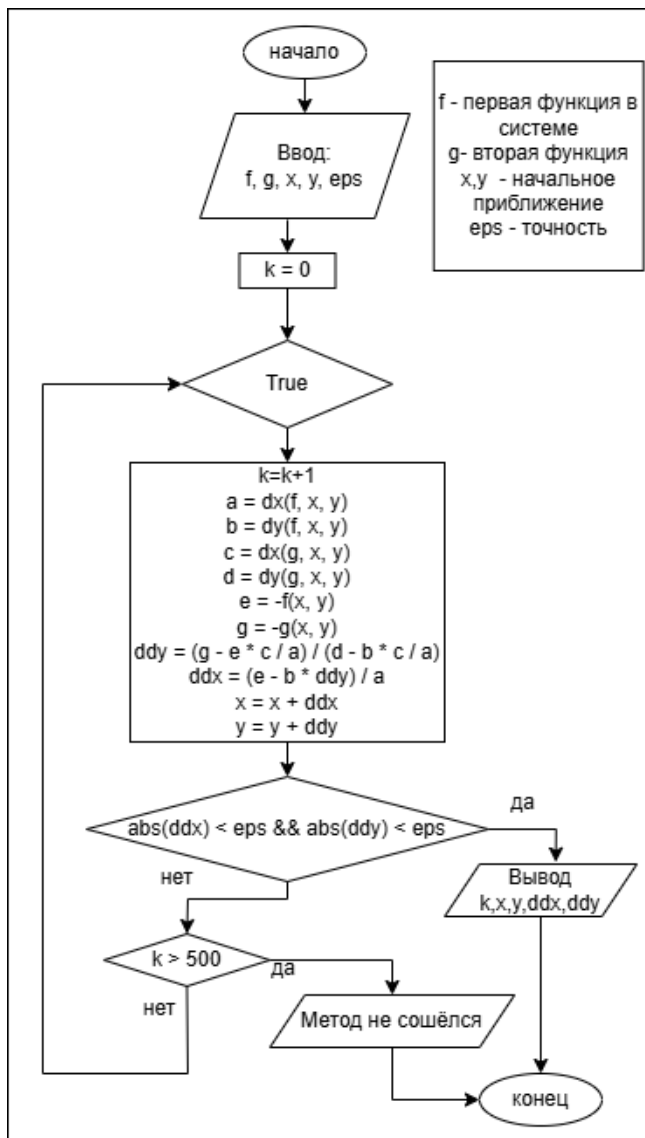
def newton(f, g, x, y, eps):
    k=0
    while (True):
        k+=1
        a, b = [dx(f, x, y), dy(f, x, y)]
        c, d = [dx(g, x, y), dy(g, x, y)]
        e, h = [-f(x, y), -g(x, y)]
        ddy = (h - e * c / a) / (d - b * c / a)
        ddx = (e - b * ddy) / a
        x = x + ddx
        y = y + ddy
  
```

```

if abs(ddx) < eps and abs(ddy) < eps:
    return k, x, y, ddx, ddy
if (k>500):
    return "метод не сошёлся"

```

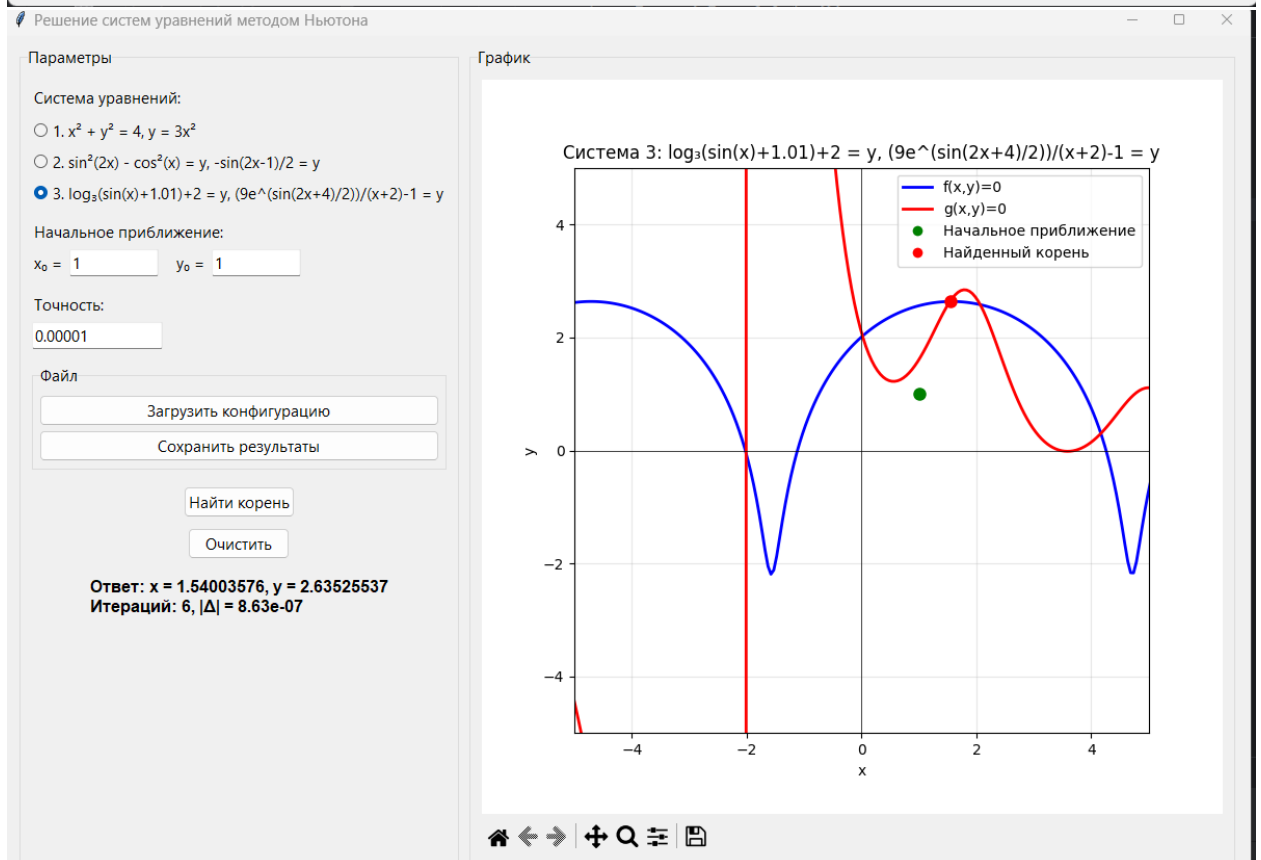
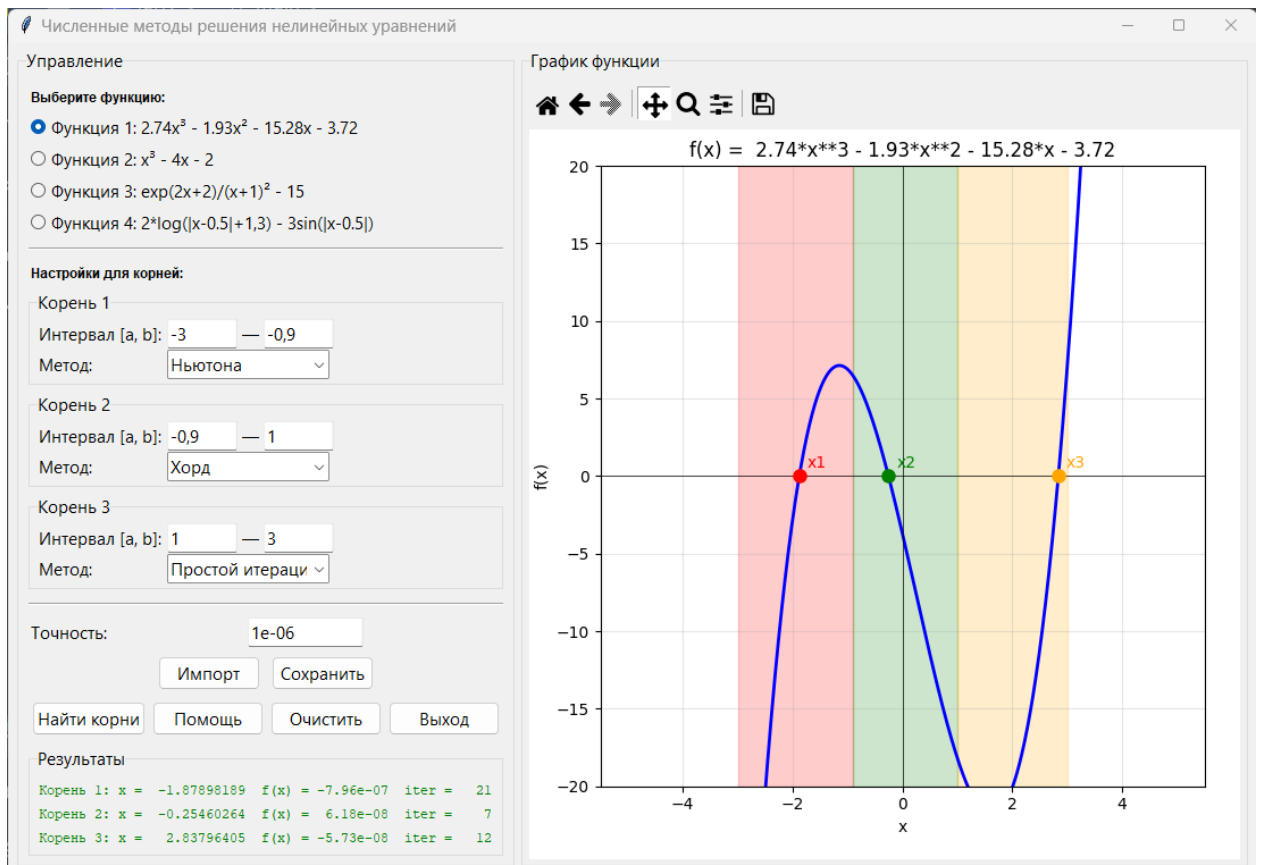
Блок-схема:



Ссылка на GitHub

<https://github.com/AndrewBond-005/itmo/tree/main/%D0%B2%D1%8B%D1%87%D0%BC%D0%B0%D1%82/lab2>

Пример работы



Вывод

В ходе лабораторной работы я изучил численные методы решения нелинейных уравнений и их систем, нашёл корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнил программную реализацию методов и нарисовал к ним блок-схемы